Семантичний аналіз програм

Мета роботи: доповнити синтаксичний аналізатор мови Krok семантичним аналізом, тобто побудовою таблиці імен, контролем оголошення та ініціації змінних і перевіркою типів у виразах та інструкціях.

1. Коротка характеристика мови

Krok є простою імперативною процедурною мовою зі статичною і суворою типізацією; мова-прототип Go, рівень складності 2. Програма складається з одного файлу: `package main`, `import "fmt"`, оголошення змінних, констант, типів і функцій. Базові типи: `int`, `float`, `bool`, `string`; крім них, є користувацькі скалярні типи (`type Celsius float`), масиви `[N]T` і структури. Параметр функції може мати тип `*T`, тоді він передається за посиланням. Інструкції: присвоєння, оголошення з ініціалізацією `:=`, `if / else if / else`, `for` з передумовою та трикомпонентний, `return`, введення й виведення через пакет `fmt`.

Тип кожного імені й виразу відомий на етапі трансляції, тому більшість помилок у програмі можна знайти до її виконання. Граматика визначає лише структуру програми; усе, що залежить від контексту (оголошеність імен, сумісність типів, l-значення тощо), перевіряє семантичний аналізатор.

2. Семантика мови

Як і в розд. 1.2 матеріалів до роботи, нижче перелічено семантичні обмеження, що їх перевіряє аналізатор. Вони взяті зі специфікації мови Krok (розділи «Типи даних і змінні», «Вирази», «Інструкції», «Декларації», «Помилки трансляції»); у дужках наведено код помилки трансляції.

1. Кожне ім'я, крім імені пакета, має бути оголошене (CE-03). Ім'я шукається від найглибшого блоку до блоку пакета; оголошення у вкладеному блоці затінює зовнішнє.

2. Повторне оголошення імені в тому самому блоці, зокрема імені параметра на верхньому рівні тіла функції, є помилкою (CE-04).

3. Типи й функції видимі в усьому файлі незалежно від порядку оголошень. Глобальні константи і змінні видимі в тілах усіх функцій, а в ініціалізаторах глобальних оголошень і в довжинах масивів лише після свого оголошення. Область видимості локального імені починається після його оголошення.

4. Змінна набуває значення в присвоюванні, в оголошенні з ініціалізатором, у введенні `fmt.Scan` або коли її адресу передано функції. Змінна без ініціалізатора має нульове значення свого типу, тому її читання до присвоєння не є помилкою.

5. Типи операндів і результатів операторів задано табл. 1. Єдиним неявним приведенням є розширення `int` до `float`. Іменовані типи еквівалентні номінально; літерал (зокрема зі знаком) набуває іменованого типу, якщо його тип збігається з базовим типом або розширюється до нього. Масиви, структури й вказівники не беруть участі в операціях (CE-05).

6. Ліва частина `=` і операнд `++`/`--` мають бути l-значеннями (CE-07), а значення праворуч від `=` має бути сумісним за присвоєнням з типом лівої частини (CE-05). Ліва частина `:=` є новим ідентифікатором (CE-17).

7. Умови `if` і `for` мають тип `bool` (CE-05).

8. Аргументи `fmt.Print`/`fmt.Println` мають скалярні типи; аргументи `fmt.Scan` є адресами змінних скалярних типів (CE-05, CE-07).

9. Виклик функції: кількість аргументів дорівнює кількості параметрів, кожен аргумент сумісний з типом параметра, параметру `*T` передається `&x` з `x` типу `T` (CE-08). Виклик функції без результату не може бути значенням (CE-15), а інструкцією-виразом може бути лише виклик (CE-17).

10. `return` без виразу допустимий лише у функції без результату, з виразом — лише у функції з результатом (CE-09); функція з результатом завершується `return` на кожному шляху виконання.

11. Довжина масиву є додатною константою типу `int`, константний індекс не виходить за межі масиву (CE-11). Ініціалізатор константи є константним виразом (CE-10), ініціалізатор глобальної змінної теж (CE-16). Ділення на нуль у константному виразі, переповнення та від'ємний цілий показник степеня є помилками трансляції (CE-10), а поза ним — помилками часу виконання (RE-01, RE-02, RE-05).

12. Явне приведення `T(x)` допустиме лише між числовими типами або між типами з однаковим базовим типом (CE-06).

13. Поля структури унікальні, звернення до неіснуючого поля і рекурсивне включення структури в себе є помилками (CE-12); програма має функцію `main` без параметрів і результату (CE-13) та імпортує `"fmt"`, якщо використовує введення-виведення (CE-14).

Таблиця 1 — Типи результатів бінарних операторів (функція baseOp; «—» означає помилку CE-05)

Оператори	**int, int**	**int, float**	**float, float**	**string, string**	**bool, bool**
+	int	float	float	string	—
- * / **	int	float	float	—	—
%	int	—	—	—	—
< <= > >=	bool	bool	bool	bool	—
== !=	bool	bool	bool	bool	bool
&& ||	—	—	—	—	bool

Два пункти семантики прикладу з матеріалів до роботи в Krok мають іншу реакцію. За п. 5 прикладу використання змінної, що не набула значення, є помилкою; у Krok така змінна має нульове значення, тому аналізатор фіксує й перевіряє факт ініціації (розд. 2.3 матеріалів), але видає попередження, як запропоновано в розд. 3.2.2 матеріалів. За п. 11 прикладу ділення на нуль є помилкою; у Krok це помилка RE-01 часу виконання, тому ділення на константний нуль поза константним виразом теж дає попередження, а в константному виразі — помилку CE-10.

Таблиця 2 — Помилки трансляції, що їх виявляє семантичний аналізатор (зі специфікації мови)

Код	**Помилка**
CE-03	використання неоголошеного імені
CE-04	повторне оголошення імені в тому самому блоці
CE-05	невідповідність типів: операнди оператора, присвоєння, ініціалізація, умова if/for, аргументи Print/Scan, кількість виразів у var
CE-06	недопустиме явне приведення типів
CE-07	присвоєння, ++/--, & чи введення для виразу, що не є l-значенням (константи, результату виклику тощо)
CE-08	невідповідність кількості або типів аргументів виклику; виклик імені, що не є функцією чи типом
CE-09	return без виразу у функції з результатом, з виразом — у функції без результату; відсутність return на деякому шляху виконання
CE-10	значення літерала поза множиною значень типу; помилка обчислення константного виразу
CE-11	довжина масиву не є додатною константою типу int; зайві елементи в літералі масиву; константний

	індекс поза межами
CE-12	повторне або неіснуюче поле структури; рекурсивне включення структури в себе
CE-13	ім'я пакета відмінне від main; відсутність функції main або неправильна її сигнатура
CE-14	використання fmt без імпорту; імпорт шляху, відмінного від "fmt"; повторний імпорт
CE-15	використання виклику функції без результату як значення
CE-16	ініціалізатор глобальної змінної не є константним виразом
CE-17	ліва частина := не є ідентифікатором; := в інструкції наступного кроку for; інструкція-вираз не є викликом функції

3. Лексичний і синтаксичний аналізатори

Використано лексичний аналізатор `krok_lex.py` з роботи КП 1 (метод діаграми станів) і синтаксичний аналізатор з роботи КП 2 (рекурсивний спуск). Лексичний аналізатор будує таблицю розбору `tableOfSymb` і таблицю констант `tableOfConst`; за токеном літерала в таблиці констант семантичний аналізатор визначає його тип (розд. 3.1.1 матеріалів).

Відповідно до розд. 2 матеріалів, семантичний аналіз виконується в процесі синтаксичного: `krok_sem.py` є парсером КП 2, функції якого доповнено семантичними діями. Граматика й порядок викликів функцій розбору не змінилися (56 функцій з декоратором `@rule`), тож множина синтаксично правильних програм та сама. Це перевіряється в кожному тесті: синтаксичний вердикт порівнюється з вердиктом парсера КП 2.

4. Програмна реалізація семантичного аналізатора

4.1. Таблиця імен і контроль оголошень

Таблиця змінних методички `tableOfVar = {id: (indx, type, value)}` описує один блок. У Krok блоків багато (блок пакета, функції, неявний блок `for`, вкладені блоки), тому таблицю імен побудовано за зразком `forTabNames.py`: це словник контекстів, кожен з яких посилається на охоплюючий через `declIn`. Як у таблиці ідентифікаторів з посібника Медведєвої (ч. 2, рис. 2.36), для кожного блока зберігається рівень, а для імені — номер у блоці, клас і тип. Контекст функції називається її іменем, вкладений блок — `<охоплюючий>.<номер>`, наприклад `main.2.1`.

Записи імен винесено у вкладений словник `names`, бо в `forTabNames.py` службовий ключ `declIn` лежить поруч з іменами, і змінна з ім'ям `declIn` зламала б таблицю. Значенням змінної є `undefined` або `assigned` (розд. 2.3 матеріалів), константи — її обчислене значення; в останньому полі функції зберігаються типи параметрів, структури — словник полів. Python-словник є перемішаною таблицею з хеш-адресацією (Медведєва, ч. 2, розд. 2.1.2), тому пошук у блоці не залежить від кількості імен.

```
def insertName(c, name, numLine, kind, typ, val, extra='-'):
    """Додати ім'я до контексту c з контролем унікальності оголошення (insert з forTabNames.py)."""
    names = tabName[c]['names']
    if name in names:
        failSem('redeclared', numLine, 'повторне оголошення {0} в {1}'.format(name, c))
    names[name] = (len(names), kind, str(typ), val, extra)

def findName(name, c, contexts=None):
    """Пошук імені від контексту c до блока пакета (findName з forTabNames.py).
    Повертає (контекст, атрибути, переглянуті контексти); якщо ім'я не знайдено - контекст None."""
    contexts = [] if contexts is None else contexts
    contexts.append(c)
    attr = tabName[c]['names'].get(name)
    if attr is not None:
        return c, attr, contexts
    if c == 'univ':
        return None, None, contexts
    return findName(name, tabName[c]['declIn'], contexts)

def getTypeVar(name):
    """Тип імені або 'undeclared_variable', якщо його не оголошено (розд. 2.2 методички)."""
    c, attr, _ = findName(name, cxt)
    return 'undeclared_variable' if c is None else attr[2]

def isInitVar(name, c):
    """Чи набула змінна значення (розд. 2.3 методички)."""
    try:
        return tabName[c]['names'][name][3] == 'assigned'
    except KeyError:
        return 'undeclared_variable'

def setAssigned(lvalue):
    """Зафіксувати факт ініціації змінної: 'assigned' замість 'undefined' (присвоювання, введення,
        &x)."""
    if lvalue is not None:
        c, name = lvalue
        idx, kind, typ, val, extra = tabName[c]['names'][name]
        if val == 'undefined':
            tabName[c]['names'][name] = (idx, kind, typ, 'assigned', extra)
```

Унікальність оголошення перевіряє `insertName()`: ім'я шукається лише в поточному контексті, тому затінення у вкладеному блоці дозволене. Неоголошене ім'я виявляє `getTypeVar()`, що повертає тип `undeclared_variable` (розд. 2.2 матеріалів). `setAssigned()` викликається при присвоюванні, введенні та передачі `&x`, а `isInitVar()` — щоразу, коли змінна використовується як значення.

4.2. Два проходи

Однопрохідний рекурсивний спуск бачить лише імена, оголошені вище, а в Krok функції можуть викликати одна одну незалежно від порядку оголошень. Тому аналіз виконується за два проходи по таблиці розбору:

– попередній перегляд `registerTopNames()` реєструє імена блока пакета в порядку оголошення з позначкою `pending`;

– прохід 1 опрацьовує оголошення блока пакета: типи, константи, глобальні змінні та сигнатури функцій; тіла функцій пропускаються до парної фігурної дужки. Якщо означення типу чи функції знадобилося раніше, ніж до нього дійшов розбір, `resolveTop()` розбирає це оголошення на вимогу і повертається назад; позначка `resolving` дозволяє виявити циклічні означення (`type A B`, `type B A`);

– після проходу 1 перевіряються функція `main` і рекурсивне включення структур;

– прохід 2 аналізує тіла функцій, коли всі імена блока пакета вже відомі.

4.3. Перевірка типів виразів

Як у розд. 3 матеріалів, функції розбору виразів повертають тип розібраного виразу: рядок `int`, `[3]float`, `Point`, `*int` тощо. Тип масиву записується з обчисленою довжиною, тому `[Size]int` і `[5]int` дають однаковий рядок, і структурна еквівалентність неіменованих масивів зводиться до порівняння рядків. Повертається об'єкт класу `Ty`, похідного від `str`, тож порівняння `t == 'int'` працюють як у методичці, а додаткові атрибути зберігають значення константного виразу (`val`), ознаку літерала (`lit`), змінну, якій належить l-значення (`lvalue`), і ознаку виклику (`call`).

Тип результату бінарного оператора обчислює `getTypeOp(lType, op, rType)`, як у розд. 3.1.2 матеріалів. Для присвоювання функцію доповнено альтернативою `op == '='`, що повертає `void` або `type_error` (розд. 3.2.1). Перед застосуванням оператора функція `binary()` перевіряє, що обидва операнди є значеннями, і, якщо вони константні, обчислює значення виразу.

```
def getTypeOp(lType, op, rType):
    """Тип результату застосування бінарного оператора op до операндів типів lType і rType
    (розд. 3.1.2 методички); для присвоювання '=' - 'void' або 'type_error' (розд. 3.2.1
        методички)."""
    if op == '=':
        return 'void' if assignable(rType, lType) else 'type_error'
    named = None
    if isNamed(lType) or isNamed(rType):          # номінальна еквівалентність іменованих типів
        if lType == rType or (isNamed(lType) and literalFits(rType, lType)):
            named = lType
        elif isNamed(rType) and literalFits(lType, rType):
            named = rType
        else:
            return 'type_error'
        lU = rU = underlying(named)
    else:
        lU, rU = str(lType), str(rType)
    typeRes = baseOp(lU, op, rU)
    if typeRes != 'type_error' and named and op not in REL_OPS:
        typeRes = named                           # операція над іменованим типом дає той самий тип
    return typeRes

def binary(lType, op, rType, numLine):
    """Перевірка типів бінарної операції (розд. 3.1.2 методички) та обчислення константного
        виразу."""
    useValue(lType, numLine)
    useValue(rType, numLine)
    resType = getTypeOp(lType, op, rType)
    if resType == 'type_error':
        failSem('type_error', numLine, 'оператор {0} не застосовний до операндів типів {1} і {2}'
                .format(op, lType, rType))
    val = None
    if lType.val is not None and rType.val is not None:
        val = fold(op, lType.val, rType.val, operandBase(lType, rType), numLine)
    elif op in ('/', '%') and rType.val is not None and rType.val == 0:
        # п. 11 семантики прикладу з методички: правий операнд '/' не повинен дорівнювати нулю
        warnSem(numLine, 'ділення на нуль - під час виконання виникне помилка RE-01')
    return Ty(resType, val=val)

@rule
def parseAddExpr():                       # AddExpr = MulExpr { AddOp MulExpr }
    lType = parseMulExpr()
    while peek()[1] == 'add_op':
        numLine, op, _ = getSymb()
        expect(op)
        lType = binary(lType, op, parseMulExpr(), numLine)
    return lType
```

4.4. Перевірка типів інструкцій

Функції розбору інструкцій повертають `void`, якщо перевірка типів завершилась успіхом. Тип списку інструкцій обчислюється за правилом з розд. 3.2.3 матеріалів (`void; void` дає `void`), умова циклу перевіряється так само, як для `while` у матеріалах. Ознака `term` показує, що інструкція завершальна: це `return`, блок із завершальним списком або `if` з гілкою `else`, де всі гілки завершальні. За нею перевіряється наявність `return` на кожному шляху виконання функції. Виведення перевіряється так, як запропоновано для `parseWrite()`: аргументи мають бути значеннями скалярних типів.

```
@rule
def parseStatementList():                # StatementList = [ Statement StatementTail ]
    lex, tok = peek()
    if lex is None or not startsStatement(lex, tok):
        return Ty('void', empty=True)
    s1 = parseStatement()
    s2 = parseStatementTail()
    # тип послідовності інструкцій Stmnt1; Stmnt2 (розд. 3.2.3 методички)
    if s1 == 'void' and s2 == 'void':
        return Ty('void', term=s1.term if s2.empty else s2.term)
    return Ty('type_error')

@rule
def parseForStmt():                      # ForStmt = for ForHeader Block
    numLine = getSymb()[0]
    expect('for')
    openContext()                        # неявний блок інструкції for
    condType = parseForHeader(numLine)
    body = parseBlock()
    closeContext()
    # if type(BoolExpr) == 'bool' then type(Stmnt) else type_error (розд. 3.2.3 методички)
    return Ty(body if condType == 'bool' else 'type_error')

@rule
def parseOutputCall():                   # OutputCall = ( 'Print' | 'Println' ) '(' [
    ExpressionList ] ')'
    numLine, lex, _ = getSymb()
    expect(lex); expect('(')
    args = parseExpressionList() if peek()[0] != ')' else []
    expect(')')
    for i, a in enumerate(args, 1):      # parseWrite - 'void', якщо аргументи - значення скалярних
        типів
        useValue(a, numLine)
        if not isScalar(a):
            failSem('type_error', numLine, 'аргумент {0} fmt.{1} має тип {2}, а не
                скалярний'.format(i, lex, a))
    return Ty('void')
```

Інструкції присвоєння, оголошення `:=`, інкременту та виклику починаються з виразу, тому `parseSimpleStmt()` спершу розбирає вираз і лише за наступною лексемою з'ясовує вид інструкції. Перед цим аналізатор переглядає на один запис уперед: якщо за ідентифікатором стоїть `:=`, це нове ім'я, і шукати його в таблиці не можна.

```
@rule
def parseSimpleStmt():                   # SimpleStmt = Expression [ AssignOp Expression | IncDecOp
    ]
    """Повертає (форма, тип): форма - 'define' (x := e), 'assign', 'incdec' або 'expr' (вираз)."""
    numLine, lex, tok = getSymb()
    # оголошення з ініціалізацією: ліва частина - нове ім'я, тому його не шукаємо в таблиці імен
    if tok == 'id' and numRow < len_tableOfSymb and tableOfSymb[numRow + 1][1] == ':=':
        parseToken(None, 'id'); expect(':=')
        e = parseExpression()
        useValue(e, numLine)
        if str(e).startswith('*'):
            failSem('type_error', numLine, 'вказівник не можна зберегти в змінній {0}'.format(lex))
```

```
        declare(lex, numLine, 'var', str(e), 'assigned')
        return 'define', e
    r0 = numRow
    left = parseExpression()
    leftText = srcText(r0, numRow)
    op = peek()[0]
    if op == ':=':
        failSem('simple_stmt', numLine, 'ліва частина := ({0}) не є
            ідентифікатором'.format(leftText))
    if op in ('=', '++', '--') and left.lvalue is None:
        what = {'const': ' (константа)', 'func': " (ім'я функції)", 'type': " (ім'я
            типу)"}.get(left.kind, '')
        if left.kind == 'param' and str(left).startswith('*'):
            what = ' (параметр-вказівник без розіменування)'
        failSem('not_lvalue', numLine, '{0}{1} не є l-значенням'.format(leftText, what))
    if op == '=':
        expect('=')
        right = parseExpression()
        useValue(right, numLine)
        # перевірка типів інструкції присвоювання (розд. 3.2.1 методички)
        if getTypeOp(left, '=', right) == 'type_error':
            failSem('type_error', numLine, '{0} типу {1} не можна присвоїти значення типу {2}'
                    .format(leftText, left, right))
        setAssigned(left.lvalue)
        return 'assign', left
    if op in ('++', '--'):
        expect(op)
        if underlying(left) not in NUMERIC:
            failSem('type_error', numLine, 'оператор {0} не застосовний до значення типу
                {1}'.format(op, left))
        useValue(left, numLine)
        setAssigned(left.lvalue)
        return 'incdec', left
    return 'expr', left
```

4.5. Повідомлення про помилки

Перша виявлена семантична помилка завершує аналіз (як `failParse()` у КП 2). Функція `failSem(kind, numLine, message)` виводить номер рядка, опис і код помилки трансляції за специфікацією, а код завершення дорівнює 100 + номер помилки (наприклад, 105 для CE-05). Попередження `warnSem()` аналіз не зупиняють. Синтаксичні помилки, як і раніше, повідомляє `failParse()` (CE-02), лексичні — лексичний аналізатор (CE-01). Після успішного аналізу друкується таблиця імен.

5. План тестування

Таблиця 3 — План тестування

№	Тип випробування	Очікуваний результат (кількість тестів)	Усього
1	таблиця імен, унікальність оголошення	CE-04 (6); успішний аналіз (2)	8
2	неоголошене ім'я, області видимості	CE-03 (7); успішний аналіз (6)	13
3	фіксація та перевірка ініціації	успішний аналіз (7)	7
4	перевірка типів виразів	CE-05 (17); успішний аналіз (3)	20
5	перевірка типів інструкцій	CE-05 (14)	14
6	приведення типів	успішний аналіз (1); CE-06 (3)	4
7	l-значення	CE-07 (5)	5
8	виклики функцій	CE-08 (6); успішний аналіз (1)	7
9	інструкція return	CE-09 (4); успішний аналіз (1)	5
10	константні вирази	CE-10 (6); успішний аналіз (1)	7
11	масиви	CE-11 (5); успішний аналіз (1)	6
12	структури та іменовані типи	CE-12 (6); успішний аналіз (1)	7
13	структура програми, імпорт	CE-13 (3); CE-14 (3)	6

14	значення виклику, прості інструкції	CE-15 (3); CE-16 (2); CE-17 (4); успішний аналіз (1)	10
15	некоректні фрагменти зі специфікації	CE-05 (2); CE-07 (1); CE-04 (1); CE-09 (1); CE-11 (1); CE-06 (1)	7
16	коректні програми	успішний аналіз (10)	10
17	лексичні та синтаксичні помилки	CE-02 (7); CE-01 (1)	8

Для кожного тесту перевіряється код помилки, а для успішних — кількість попереджень. Окремо перевіряється, що серед очікуваних результатів є кожен код CE-01 ... CE-17, і що синтаксичний вердикт збігається з вердиктом парсера КП 2. Група 15 містить усі семантичні фрагменти з таблиці некоректних програм специфікації, групи 16 і 17 — приклади програм специфікації. Усього 144 тестів, програма `tests/run_tests.py`.

6. Протокол тестування

Тест № 1 (1 таблиця імен, унікальність оголошення): локальна змінна з ім'ям параметра.

```
 1  package main
 2
 3  import "fmt"
 4
 5  func g(a int) {
 6      var a int
 7      a = 1
 8  }
 9
10  func main() {
11
12  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic ERROR:
     В рядку 6 повторне оголошення a в g (CE-04).
Semantic: Аварійне завершення програми з кодом 104
```

Оцінка: параметр і змінні верхнього рівня тіла функції належать одному блоку `g`, тому повторне оголошення виявлено, а повідомлення називає блок.

Тест № 2 (2 неоголошене ім'я, області видимості): змінна блоку поза блоком.

```
 1  package main
 2
 3  import "fmt"
 4
 5
 6  func main() {
 7      {
 8          z := 1
 9          fmt.Println(z)
10      }
11      fmt.Println(z)
12  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic ERROR:
     В рядку 11 ім'я z не оголошено (переглянуто контексти ['main', 'univ']) (CE-03).
Semantic: Аварійне завершення програми з кодом 103
```

Оцінка: після виходу з блоку його контекст більше не переглядається; у повідомленні видно ланцюжок контекстів, за яким ішов пошук.

Тест № 3 (2 неоголошене ім'я, області видимості): взаємна рекурсія функцій.

```
 1  package main
 2
 3  import "fmt"
 4
 5  func isEven(n int) bool {
 6      if n == 0 {
 7          return true
 8      }
 9      return isOdd(n - 1)
10  }
11  func isOdd(n int) bool {
12      if n == 0 {
13          return false
14      }
15      return isEven(n - 1)
16  }
17
18  func main() {
19      fmt.Println(isEven(10))
20  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Parser: Синтаксичний аналіз завершився успішно
Semantic: Семантичний аналіз завершився успішно
Таблиця імен: контекст (охоплюючий контекст, рівень блока); ім'я, номер у блоці, клас, тип,
    значення, параметри
univ (declIn=-, рівень=0)
  isEven      0  func   bool         -          (int)
  isOdd       1  func   bool         -          (int)
  main        2  func   void         -          ()
isEven (declIn=univ, рівень=1)
  n           0  param  int          assigned   -
isOdd (declIn=univ, рівень=1)
  n           0  param  int          assigned   -
```

Оцінка: `isOdd` викликається раніше, ніж оголошена; сигнатури всіх функцій відомі після проходу 1, тому виклик перевірено. Таблиця показує сигнатури й параметри.

Тест № 4 (3 фіксація та перевірка ініціації): читання до присвоєння: попередження.

```
 1  package main
 2
 3  import "fmt"
 4
 5
 6  func main() {
 7      var n int
 8      fmt.Println(n)
 9  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic WARNING: в рядку 8 змінна n використовується до присвоєння їй значення - буде використано
    нульове значення типу int
Parser: Синтаксичний аналіз завершився успішно
Semantic: Семантичний аналіз завершився успішно (попереджень: 1)
Таблиця імен: контекст (охоплюючий контекст, рівень блока); ім'я, номер у блоці, клас, тип,
    значення, параметри
univ (declIn=-, рівень=0)
  main        0  func   void         -          ()
main (declIn=univ, рівень=1)
  n           0  var    int          undefined  -
```

Оцінка: змінна n має значення undefined, тому при її читанні видано попередження; за специфікацією буде використано нульове значення, тож програма коректна.

Тест № 5 (3 фіксація та перевірка ініціації): fmt.Scan(&n) ініціює змінну.

```
 1  package main
 2
 3  import "fmt"
 4
 5
 6  func main() {
 7      var n int
 8      fmt.Scan(&n)
 9      fmt.Println(n)
10  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Parser: Синтаксичний аналіз завершився успішно
Semantic: Семантичний аналіз завершився успішно
Таблиця імен: контекст (охоплюючий контекст, рівень блока); ім'я, номер у блоці, клас, тип,
    значення, параметри
univ (declIn=-, рівень=0)
  main        0  func   void         -          ()
main (declIn=univ, рівень=1)
  n           0  var    int          assigned   -
```

Оцінка: введення встановило для n значення assigned, попередження немає.

Тест № 6 (4 перевірка типів виразів): Celsius + Fahrenheit.

```
 1  package main
 2
 3  import "fmt"
 4
 5  type Celsius float
 6  type Fahrenheit float
 7
 8  func main() {
 9      var c Celsius = 1
10      var f Fahrenheit = 2
11      x := c + f
12      fmt.Println(x)
13  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic ERROR:
     В рядку 11 оператор + не застосовний до операндів типів Celsius і Fahrenheit (CE-05).
Semantic: Аварійне завершення програми з кодом 105
```

Оцінка: іменовані типи еквівалентні номінально: обидва мають базовий тип `float`, але додавати їх не можна.

Тест № 7 (4 перевірка типів виразів): c*9/5 + 32: літерали набувають типу Celsius.

```
 1  package main
 2
 3  import "fmt"
 4
 5  type Celsius float
 6  type Fahrenheit float
 7
 8  func main() {
 9      var c Celsius = 36.6
10      c = c*9/5 + 32
11      fmt.Println(c)
12  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Parser: Синтаксичний аналіз завершився успішно
Semantic: Семантичний аналіз завершився успішно
Таблиця імен: контекст (охоплюючий контекст, рівень блока); ім'я, номер у блоці, клас, тип,
    значення, параметри
univ (declIn=-, рівень=0)
  Celsius     0  type   float        -          -
  Fahrenheit  1  type   float        -          -
  main        2  func   void         -          ()
main (declIn=univ, рівень=1)
  c           0  var    Celsius      assigned   -
```

Оцінка: літерали 9, 5, 32 неявно набули типу `Celsius`, тому результат має тип `Celsius` і присвоєння коректне.

Тест № 8 (5 перевірка типів інструкцій): умова трикомпонентного for типу int.

```
 1  package main
 2
 3  import "fmt"
 4
 5
 6  func main() {
 7      for i := 0; i; i++ {
 8      }
 9  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic ERROR:
     В рядку 7 умова for має тип int, а не bool (CE-05).
Semantic: Аварійне завершення програми з кодом 105
```

Оцінка: умова має тип `int`, а потрібен `bool`; числа не інтерпретуються як логічні значення.

Тест № 9 (6 приведення типів): float(true).

```
 1  package main
 2
 3  import "fmt"
 4
 5
 6  func main() {
 7      fmt.Println(float(true))
 8  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic ERROR:
     В рядку 7 недопустиме явне приведення типу bool до типу float (CE-06).
Semantic: Аварійне завершення програми з кодом 106
```

Оцінка: приведення між `bool` і числовими типами недопустиме.

Тест № 10 (7 l-значення): присвоєння параметру-вказівнику.

```
 1  package main
 2
 3  import "fmt"
 4
 5  func f(p, q *int) {
 6      p = q
 7  }
 8
 9  func main() {
10
11  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic ERROR:
     В рядку 6 p (параметр-вказівник без розіменування) не є l-значенням (CE-07).
Semantic: Аварійне завершення програми з кодом 107
```

Оцінка: параметр-вказівник без розіменування не є l-значенням; змінити можна лише `*p`.

Тест № 11 (8 виклики функцій): &float для параметра *int.

```
 1  package main
 2
 3  import "fmt"
 4
 5  func swap(a, b *int) {
 6      t := *a
 7      *a = *b
 8      *b = t
 9  }
10
11  func main() {
12      var x float
13      y := 1
14      swap(&x, &y)
15  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic ERROR:
         В рядку 14 аргумент 1 функції swap має бути адресою змінної типу int (&x), а не значенням
      типу *float (CE-08).
Semantic: Аварійне завершення програми з кодом 108
```

Оцінка: для параметрів-вказівників неявне розширення не діє: тип `*float` не збігається з `*int`.

Тест № 12 (9 інструкція return): немає return у гілці.

```
 1  package main
 2
 3  import "fmt"
 4
 5  func f(a bool) int {
 6      if a {
 7          return 1
 8      }
 9  }
10
11  func main() {
12
13  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic ERROR:
         В рядку 5 функція f з результатом int не завершується інструкцією return на кожному шляху
      виконання (CE-09).
Semantic: Аварійне завершення програми з кодом 109
```

Оцінка: список інструкцій тіла закінчується `if` без `else`, тобто не є завершальним.

Тест № 13 (10 константні вирази): переповнення в константному виразі.

```
 1  package main
 2
 3  import "fmt"
 4
 5  const B = 9223372036854775807 + 1
 6
 7  func main() {
 8
 9  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic ERROR:
     В рядку 5 переповнення int у константному виразі 9223372036854775807 + 1 (CE-10).
Semantic: Аварійне завершення програми з кодом 110
```

Оцінка: значення константного виразу обчислюється під час трансляції, і переповнення `int` виявлено до виконання програми.

Тест № 14 (11 масиви): константний індекс поза межами.

```
 1  package main
 2
 3  import "fmt"
 4
 5
 6  func main() {
 7      var a [3]int
 8      a[3] = 1
 9  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic ERROR:
     В рядку 8 константний індекс 3 поза межами [0, 2] (CE-11).
Semantic: Аварійне завершення програми з кодом 111
```

Оцінка: індекс є константою, тому вихід за межі `[0, 2]` виявлено статично.

Тест № 15 (12 структури та іменовані типи): рекурсивне включення структури.

```
 1  package main
 2
 3  import "fmt"
 4
 5  type Node struct {
 6      next Node
 7  }
 8
 9  func main() {
10
11  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic ERROR:
     В рядку 5 структура Node рекурсивно включає сама себе (CE-12).
Semantic: Аварійне завершення програми з кодом 112
```

Оцінка: розмір значення такої структури був би нескінченним; перевірку виконано після проходу 1, коли відомі всі типи.

Тест № 16 (13 структура програми, імпорт): fmt без import.

```
 1  package main
 2
 3  func main() {
 4      fmt.Println(1)
 5  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic ERROR:
     В рядку 4 використання fmt без import "fmt" (CE-14).
Semantic: Аварійне завершення програми з кодом 114
```

Оцінка: використання пакета без оголошення імпорту виявлено вже на першій інструкції.

Тест № 17 (14 значення виклику, прості інструкції): x := nop(): виклик без результату.

```
 1  package main
 2
 3  import "fmt"
 4
 5  func nop() {
 6  }
 7
 8  func main() {
 9      x := nop()
10      fmt.Println(x)
11  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic ERROR:
     В рядку 9 виклик функції без результату використано як значення (CE-15).
Semantic: Аварійне завершення програми з кодом 115
```

Оцінка: виклик функції без результату не має значення, тому його не можна присвоїти змінній.

Тест № 18 (14 значення виклику, прості інструкції): := у наступному кроці for.

```
1  package main
2
3  import "fmt"
4
5
6  func main() {
7      for i := 0; i < 3; j := 1 {
8      }
9  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic ERROR:
     В рядку 7 оголошення := в інструкції наступного кроку for (CE-17).
Semantic: Аварійне завершення програми з кодом 117
```

Оцінка: оголошення в інструкції наступного кроку недопустиме, хоча граматика дозволяє тут будь-яку просту інструкцію.

Тест № 19 (16 коректні програми): 07_scope_shadowing.krok.

```
1  // Області видимості та затінення
2  package main
3
4  import "fmt"
5
6  var x int = 1 // глобальна
7
8  func show() {
9      fmt.Println("global x =", x) // глобальна x: 1
10 }
11
12 func main() {
13     fmt.Println(x) // 1
14     x := "local"    // затінює глобальну x у тілі main
15     {
16         x := 3.5 // затінює локальну x у вкладеному блоці
17         fmt.Println(x)
18     }
19     fmt.Println(x) // local
20     for i := 0; i < 2; i++ {
21         x := i // нова змінна на кожне виконання тіла
22         fmt.Println(x)
23     }
24     show()
25 }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Parser: Синтаксичний аналіз завершився успішно
Semantic: Семантичний аналіз завершився успішно
Таблиця імен: контекст (охоплюючий контекст, рівень блока); ім'я, номер у блоці, клас, тип,
    значення, параметри
univ (declIn=-, рівень=0)
  x           0  var    int          assigned   -
  show        1  func   void         -          ()
  main        2  func   void         -          ()
main (declIn=univ, рівень=1)
  x           0  var    string       assigned   -
main.1 (declIn=main, рівень=2)
  x           0  var    float        assigned   -
main.2 (declIn=main, рівень=2)
  i           0  var    int          assigned   -
main.2.1 (declIn=main.2, рівень=3)
  x           0  var    int          assigned   -
```

Оцінка: чотири змінні `x` різних типів лежать у різних контекстах: `univ`, `main`, `main.1`, `main.2.1`; кожне використання знайшло найближче оголошення.

Тест № 20 (16 коректні програми): 08_runtime_errors.krok.

```
 1  // Помилки часу виконання
 2  package main
 3
 4  import "fmt"
 5
 6  func depth(n int) int {
 7      return depth(n+1) + 1 // RE-06: переповнення стека
 8  }
 9
10  func main() {
11      var a [3]int
12      var i, z int
13      fmt.Scan(&i, &z)
14      fmt.Println(a[i]) // RE-04 при i < 0 або i > 2
15      fmt.Println(10 / z) // RE-01 при z == 0
16      big := 9223372036854775807
17      fmt.Println(big + 1) // RE-02: переповнення int
18      fmt.Println(depth(0))
19  }
```

```
>python krok_sem.py test.krok
Lexer: Лексичний аналіз завершено успішно
Semantic: прохід 1 - оголошення блока пакета (типи, константи, змінні, сигнатури функцій)
Semantic: прохід 2 - тіла функцій
Semantic WARNING: в рядку 14 змінна a використовується до присвоєння їй значення - буде використано
    нульове значення типу [3]int
Parser: Синтаксичний аналіз завершився успішно
Semantic: Семантичний аналіз завершився успішно (попереджень: 1)
Таблиця імен: контекст (охоплюючий контекст, рівень блока); ім'я, номер у блоці, клас, тип,
    значення, параметри
univ (declIn=-, рівень=0)
  depth       0  func   int          -          (int)
  main        1  func   void         -          ()
depth (declIn=univ, рівень=1)
  n           0  param  int          assigned   -
main (declIn=univ, рівень=1)
  a           0  var    [3]int       undefined  -
  i           1  var    int          assigned   -
  z           2  var    int          assigned   -
  big         3  var    int          assigned   -
```

Оцінка: масив `a` читається з нульовими значеннями, тому програма коректна, але аналізатор попереджає про це.

Повний протокол автоматичного тестування:

```
>python tests/run_tests.py
ok    1  x := 1 і x := 2 в одному блоці                CE-04
ok    1  var a, a int                                  CE-04
ok    1  глобальна змінна і функція з тим самим ім'ям  CE-04
ok    1  два оголошення типу T                         CE-04
ok    1  повторне ім'я параметра                       CE-04
ok    1  локальна змінна з ім'ям параметра             CE-04
ok    1  затінення у вкладеному блоці                  ok, попереджень 0
ok    1  затінення глобальної змінної                  ok, попереджень 0
ok    2  неоголошена змінна y = 5                      CE-03
ok    2  неоголошена змінна у виразі                   CE-03
ok    2  змінна блоку поза блоком                      CE-03
ok    2  змінна циклу після циклу                      CE-03
ok    2  неоголошений тип                              CE-03
ok    2  константа до свого оголошення                 CE-03
ok    2  x := x + 1: праворуч зовнішня x               ok, попереджень 0
ok    2  взаємна рекурсія функцій                      ok, попереджень 0
ok    2  тип оголошено після використання              ok, попереджень 0
ok    2  ініціалізатор з типом, оголошеним нижче       ok, попереджень 0
ok    2  тип нижче, його довжина: константа між ними   ok, попереджень 0
ok    2  глобальна змінна нижче функції видима в тілі  ok, попереджень 0
ok    2  довжина масиву: константа, оголошена нижче    CE-03
ok    3  читання до присвоєння: попередження           ok, попереджень 1
ok    3  fmt.Scan(&n) ініціює змінну                   ok, попереджень 0
ok    3  присвоювання ініціює змінну                   ok, попереджень 0
ok    3  передача &n ініціює змінну                    ok, попереджень 0
ok    3  присвоєння елементу ініціює масив             ok, попереджень 0
ok    3  s = s + i у циклі до присвоєння               ok, попереджень 1
ok    3  len(a) не читає елементів                     ok, попереджень 0
ok    4  int + string                                  CE-05
ok    4  string - string                               CE-05
ok    4  float % int                                   CE-05
ok    4  bool < bool                                   CE-05
ok    4  !5                                            CE-05
ok    4  -"s"                                          CE-05
ok    4  1 && true                                     CE-05
ok    4  порівняння масивів                            CE-05
ok    4  Celsius + Fahrenheit                          CE-05
ok    4  c*9/5 + 32: літерали набувають типу Celsius   ok, попереджень 0
ok    4  1 + 2.5 має тип float                         CE-05
ok    4  7 / 2 має тип int, розширюється до float      ok, попереджень 0
ok    4  індекс типу float                             CE-05
ok    4  індексування не масиву                        CE-05
ok    4  індексування рядка                            CE-05
ok    4  розіменування не вказівника                   CE-05
ok    4  len(5)                                        CE-05
ok    4  ім'я функції як значення                      CE-05
ok    4  ім'я типу як значення                         CE-05
ok    4  x / 0: попередження про RE-01                 ok, попереджень 1
ok    5  умова for типу int                            CE-05
ok    5  умова трикомпонентного for типу int           CE-05
ok    5  умова for є присвоєнням                       CE-05
ok    5  присвоєння string змінній int                 CE-05
ok    5  fmt.Println масиву                            CE-05
ok    5  fmt.Scan структури                            CE-05
ok    5  s++ для string                                CE-05
ok    5  var a, b int = 1                              CE-05
ok    5  return "s" з функції int                      CE-05
ok    5  константа типу масиву                         CE-05
ok    5  вказівник у змінній                           CE-05
ok    5  елемент літерала масиву типу string           CE-05
ok    5  присвоєння Celsius <- Fahrenheit              CE-05
ok    5  присвоєння Celsius <- float                   CE-05
ok    6  int(2.9), float(3), Celsius(n)                ok, попереджень 0
ok    6  float(true)                                   CE-06
ok    6  string(5)                                     CE-06
ok    6  приведення структури Point(p)                 CE-06
ok    7  присвоєння результату виклику                 CE-07
ok    7  5++                                           CE-07
ok    7  fmt.Scan(&N) для константи                    CE-07
ok    7  присвоєння параметру-вказівнику               CE-07
ok    7  адреса літерала &3                            CE-07
ok    8  неправильна кількість аргументів             CE-08
ok    8  аргумент неправильного типу                   CE-08
ok    8  значення замість адреси для *int              CE-08
ok    8  адреса для параметра-значення                 CE-08
```

```
ok    8  &float для параметра *int                      CE-08
ok    8  виклик змінної n(1)                            CE-08
ok    8  параметр-вказівник передано далі               ok, попереджень 0
ok    9  немає return у гілці                           CE-09
ok    9  return без виразу у функції int                CE-09
ok    9  return з виразом у функції без результату      CE-09
ok    9  цикл не є завершальною інструкцією             CE-09
ok    9  if/else з return в обох гілках                 ok, попереджень 0
ok   10  const Z = 1 / 0                                CE-10
ok   10  цілий літерал > max int                        CE-10
ok   10  переповнення в константному виразі             CE-10
ok   10  від'ємний цілий показник степеня               CE-10
ok   10  ініціалізатор константи не константний         CE-10
ok   10  мінімальне int: -9223372036854775807 - 1       ok, попереджень 0
ok   10  дійсний літерал 1e400                          CE-10
ok   11  var a [0]int                                   CE-11
ok   11  довжина масиву є змінною                       CE-11
ok   11  довжина масиву типу float                      CE-11
ok   11  зайві елементи в літералі                      CE-11
ok   11  константний індекс поза межами                 CE-11
ok   11  довжина масиву: [N]int, [len(a)]float          ok, попереджень 0
ok   12  повторне поле структури                        CE-12
ok   12  неіснуюче поле                                 CE-12
ok   12  поле значення, що не є структурою              CE-12
ok   12  рекурсивне включення структури                 CE-12
ok   12  рекурсія через масив                           CE-12
ok   12  циклічне визначення типів                      CE-12
ok   12  іменований масив <- неіменований               ok, попереджень 0
ok   13  ім'я пакета не main                            CE-13
ok   13  немає функції main                             CE-13
ok   13  main з параметром                              CE-13
ok   13  fmt без import                                 CE-14
ok   13  import "os"                                    CE-14
ok   13  повторний import "fmt"                         CE-14
ok   14  x := nop(): виклик без результату              CE-15
ok   14  fmt.Println(nop())                             CE-15
ok   14  1 + nop()                                      CE-15
ok   14  глобальна змінна = виклик                      CE-16
ok   14  глобальна змінна = інша змінна                 CE-16
ok   14  інструкція-вираз x + 1                         CE-17
ok   14  інструкція-вираз int(x)                        CE-17
ok   14  a[0] := 1                                      CE-17
ok   14  := у наступному кроці for                      CE-17
ok   14  виклик функції як інструкція                   ok, попереджень 0
ok   15  var n int = 2.5                                CE-05
ok   15  if n { } для n int                             CE-05
ok   15  const N = 3 і N = 4                            CE-07
ok   15  x := 1 і x := 2 в одному блоці                 CE-04
ok   15  func f() int { if a { return 1 } }             CE-09
ok   15  var a [0]int                                   CE-11
ok   15  fmt.Println(int("5"))                          CE-06
ok   16  01_types_conversions.krok                      ok, попереджень 0
ok   16  02_if_else.krok                                ok, попереджень 0
ok   16  03_for_loops.krok                              ok, попереджень 0
ok   16  04_recursion.krok                              ok, попереджень 0
ok   16  05_reference_params.krok                       ok, попереджень 0
ok   16  06_arrays_structs.krok                         ok, попереджень 0
ok   16  07_scope_shadowing.krok                        ok, попереджень 0
ok   16  08_runtime_errors.krok                         ok, попереджень 1
ok   16  base_example.krok                              ok, попереджень 0
ok   16  nested.krok                                    ok, попереджень 0
ok   17  chained_assignment.krok                        CE-02
ok   17  chained_comparison.krok                        CE-02
ok   17  else_on_new_line.krok                          CE-02
ok   17  if_without_braces.krok                         CE-02
ok   17  leading_zero.krok                              CE-01
ok   17  local_type_decl.krok                           CE-02
ok   17  pointer_variable.krok                          CE-02
ok   17  scan_without_address.krok                      CE-02
----------------------------------------
коди помилок, що перевіряються: CE-01, CE-02, CE-03, CE-04, CE-05, CE-06, CE-07, CE-08,
    CE-09, CE-10, CE-11, CE-12, CE-13, CE-14, CE-15, CE-16, CE-17
усі 144 тестів пройдено
```

7. Висновки

Синтаксичний аналізатор КП 2 доповнено семантичним аналізом. Він будує таблицю імен з блоками різного рівня, контролює унікальність оголошень, використання неоголошених імен і

факт ініціації змінних, обчислює типи виразів і перевіряє типи інструкцій. Аналізатор виявляє всі помилки трансляції специфікації мови Krok, що стосуються семантики (CE-03 ... CE-17), і приймає всі приклади програм зі специфікації.

Найбільше роботи вимагали правила видимості. Методичка розглядає мову з одним розділом оголошень, де досить однієї таблиці й одного проходу. У Krok є вкладені блоки, затінення і функції, що викликають одна одну до оголошення, тому знадобилися контексти з `forTabNames.py` і другий прохід.

Специфікація, написана в РГР, виявилась тут основним джерелом: кожна перевірка аналізатора відповідає її пункту, а таблицю некоректних фрагментів перенесено в тести без змін. Де семантика прикладу з методички розходиться зі специфікацією (неініціалізовані змінні, ділення на нуль), механізм перевірки взято з методички, а реакцію (попередження замість помилки) — зі специфікації.

Коректність підтверджують 144 тестів за планом із табл. 3. Обчислені під час аналізу значення констант і типи виразів знадобляться на наступному етапі, при перекладі програми у внутрішнє подання.

Перелік використаних джерел

1. Стативка Ю. Семантичний аналіз: матеріали до виконання роботи КП 3. 2024. 11 с.; приклади progExample.pas, forTabNames.py.
2. Матеріали до виконання роботи «Синтаксичний аналіз методом рекурсивного спуску» (КП 2).
3. Стативка Ю. І. Формальні мови: Основні концепти і представлення [Текст]: навч. посіб. / Ю. І. Стативка. Київ: КПІ ім. Ігоря Сікорського, 2023. 87 с.
4. Медведєва В. М. Транслятори: лексичний та синтаксичний аналізатори [Текст]: навч. посіб. / В. М. Медведєва, В. А. Третяк. К.: НТУУ «КПІ», 2012. 148 с.
5. Медведєва В. М. Транслятори: внутрішнє подання програм та інтерпретація [Текст]: навч. посіб. / В. М. Медведєва, В. А. Третяк. К.: Текст, 2015. 144 с.
6. Основи розробки трансляторів: лекція 1 (Транслятори_01.2026). КПІ ім. Ігоря Сікорського, 2026.
7. Степаненко Н. Ю. Специфікація мови програмування Krok: розрахунково-графічна робота. 2026.